Continuous Assessment Summary: Advanced Database Systems (MIT 8103)

Semester First Assessment | Academic Session 2026/2027

Student: Odunsi Akeem | **Student ID:** 301907681 | **Program:** Master of Information Technology

PROJECT OVERVIEW

For this assessment, I developed a simulated Call Data Record (CDR) Analysis and Link Analysis System. The idea was to create a database system that could support security operatives and crime investigators in storing and retrieving call records, identifying frequently communicating telephone numbers, analyzing direct communication links, and detecting invalid records during data import. The core dataset contains call IDs, originating and receiving telephone numbers, timestamps, call durations, import batch IDs, and cell-site identifiers. The motivation to develop this system was born from the challenges associated with manually analyzing large volumes of CDR data, particularly when trying to identify communication patterns and relationships between telephone numbers. Such analysis can be painstaking and time-consuming, especially when the CDRs are obtained from multiple sources. The project focuses on the database components of the system rather than a complete operational application. However, it demonstrates how structured data management and querying can support CDR analysis. To ensure privacy, all records used in the project were synthetically generated to avoid exposing real subscriber information or sensitive telecommunications metadata.

PORTFOLIO BREAKDOWN

Portfolio 1: Relational Design and Schema

I set up the primary database, cdr_analysis_db, in PostgreSQL 18 using pgAdmin 4. Thereafter, I structured the database into five related tables: **telephone_numbers, import_batches, cell_sites, call_events,** and **import_rejections**. This was to ensure that the database could properly accommodate the different types of information contained in telecommunications CDRs. To maintain normalization and reduce unnecessary duplication, I separated reference data, such as telephone numbers and cell-site information, from the main call events table. The rejection table was also included to maintain a record of invalid entries during data import. The primary keys, foreign keys, and constraints are documented in the ER diagram and database schema. To test the system, I developed a Python script to process 10,000 synthetic CDR records through the import pipeline. Of these, 9,500 were successfully imported, while 500 were rejected. The rejected records consisted of 100 cases each of invalid originating numbers, incorrect timestamps, negative call durations, duplicate event IDs, and unknown cell-site identifiers. I deliberately introduced these errors to simulate data-quality problems that could occur when processing CDRs obtained from different sources and to verify that the database could identify and reject invalid records.

Portfolio 2: Query Processing & Index Optimization

For this portfolio, I wrote a series of SQL queries to identify the most frequent callers, retrieve call records associated with specific telephone numbers, and determine which pairs of numbers communicated most frequently. To evaluate the performance of these queries, I used **EXPLAIN ANALYZE** to examine their execution plans before and after creating two indexes: a single-column index on (**originating_number**) and a composite index on (**originating_number, receiving_number**). The results showed that the indexes improved the efficiency of targeted record lookups. However, I also observed that PostgreSQL continued to use sequential scans for some aggregation queries despite the availability of indexes. This helped me understand that creating an index does not necessarily mean PostgreSQL will use it, as the query planner considers factors such as table size, query structure, and the estimated cost of retrieving the required records.

Portfolio 3: Transactions and Concurrency

For this portfolio, I created a separate table called **transaction_test** to carry out transaction and concurrency experiments without interfering with the main CDR database. I started by testing **COMMIT** and **ROLLBACK** operations using two simulated account balances. A successful transaction transferred 2,000 between the accounts,

leaving the balances at 8,000 and 7,000 respectively. I then tested ROLLBACK by making another change and reversing it before committing. This restored the balances to their previous values. I also examined how PostgreSQL handles concurrent transactions by using **SELECT** ... **FOR UPDATE NOWAIT** on a row already locked by another session. As expected, PostgreSQL returned error 55P03, indicating that the requested lock could not be acquired immediately. Finally, I compared two transaction isolation levels. Under **READ COMMITTED**, I observed that a second query within the same transaction could return a different value after another session committed an update. However, under **REPEATABLE READ**, both queries returned the same value despite the update made in the other session. This experiment helped me understand how transaction isolation levels affect data consistency when multiple users access a database simultaneously.

Portfolio 4: NoSQL with MongoDB

For this portfolio, I used MongoDB 9.0 through MongoDB Compass to create a NoSQL database named **cdr_analysis_nosql**. I imported the 9,500 validated CDR records into a collection called **call_events** and developed queries to retrieve records associated with specific telephone numbers, including calls made and received. I also created aggregation pipelines to identify the most frequent callers and telephone-number pairs with the highest communication frequency. To evaluate query performance, I compared the execution plans before and after creating an index on **originating_number**. Before indexing, MongoDB performed a full **collection scan (COLLSCAN)** across all 9,500 documents. However, after indexing, the query used an **index scan (IXSCAN)** that examined only 23 keys. This demonstrated how indexing could reduce the number of records MongoDB needed to examine when retrieving specific information. The experiment also helped me understand the flexibility of document-based databases, particularly for storing and querying CDR data. However, I observed that maintaining consistent data structures and enforcing validation rules required deliberate configuration, unlike the more structured approach used in PostgreSQL.

Portfolio 5: Distributed Replica Set Setup

For this portfolio, I configured a three-node MongoDB replica set named **cdrReplicaSet**, running on ports 27101, 27102, and 27103 of my Windows computer. Node 1 initially served as the PRIMARY, while Nodes 2 and 3 operated as SECONDARY nodes. To verify replication, I inserted a synthetic record **(REPL_TEST_001)** into Node 1 and confirmed that the same record was available on both secondary nodes. I then simulated a primary-node failure by deliberately shutting down Node 1. Following the shutdown, Node 2 was automatically elected as the new PRIMARY and successfully accepted another record (**REPL_TEST_002**). When I restarted Node 1, it rejoined the replica set as a SECONDARY and synchronized the records it had missed during the downtime. I confirmed this by retrieving **REPL_TEST_002** from the recovered node. This experiment demonstrated how MongoDB handles data replication, automatic failover and recovery without losing write availability when a majority of nodes remain operational. However, since all three nodes were hosted on a single Windows computer, the experiment only simulated a distributed environment and did not test resilience against physical server failure or network disruptions between separate machines.

PRACTICAL EVIDENCE MAPPING TABLE

Portfolio	Core Artifacts & Proof
Portfolio 1	Schema DDL, ER Diagram layout, synthetic import batch & rejection counts
Portfolio 2	Raw SQL scripts, pre- and post-index EXPLAIN ANALYZE execution logs
Portfolio 3	Terminal screenshots of COMMIT, ROLLBACK, error 55P03, and isolation tests
Portfolio 4	Document structures, aggregation pipelines, COLLSCAN vs IXSCAN query plans

Portfolio	Core Artifacts & Proof
Portfolio 5	Replica set status logs, failover election traces, post-recovery sync proof

Complete scripts, screenshots, and setup instructions are organized in the project's GitHub directory under /evidence *and individual portfolio folders.*

TECHNICAL REFLECTION & SELF-ASSESSMENT

Personal Reflection

I chose a CDR dataset because it aligns with my field of work, and I have personally experienced the difficulties associated with manual CDR analysis. Additionally, CDR data provided a practical opportunity to explore database normalization, bulk data import, query optimization and NoSQL modelling within a single project. I deliberately limited the scope to call events and direct communication links to keep the system manageable without unnecessarily complicating the database design. Developing an error-rejection process also helped me understand the importance of validating records during data import rather than attempting to correct errors after they have entered the database. One of my biggest lessons came from the query optimization experiments, where I discovered that creating indexes does not automatically improve every query. PostgreSQL sometimes preferred sequential scans even after I had created indexes, particularly for aggregation queries. Similarly, comparing READ COMMITTED and REPEATABLE READ helped me better understand how transaction isolation affects data consistency when multiple sessions access the same database. Finally, configuring the MongoDB replica set and observing the automatic election of a new primary gave me practical insight into database replication, availability and recovery. However, running all three nodes on one computer also helped me appreciate the limitations of testing distributed systems in a local environment.

Project Limitations & Next Steps

Because this project ran locally on a single machine with synthetic data, it doesn't represent true enterprise production scenarios. It skips real-time high-throughput ingestion, multi-user security roles, and geographical redundancy. If I extend this project, I would test scaling with multi-million row datasets, configure Role-Based Access Control (RBAC), implement multi-hop link analysis, and deploy the MongoDB nodes across distinct physical machines or cloud instances.

References and Assessment Declarations

1. Miva University. (2026). *MIT 8103 Advanced Database Systems: First Semester Continuous Assessment, 2026/2027 Academic Session.*
2. PostgreSQL Global Development Group. *PostgreSQL 18 Documentation: Indexes, EXPLAIN and Transaction Isolation.* https://www.postgresql.org/docs/18/
3. MongoDB, Inc. *MongoDB Manual: Replica Sets, Aggregation and Indexing.* https://www.mongodb.com/docs/manual/

Project Repository: https://github.com/odunsi-svg/Odunsi_Akeem_301907681

Declaration on Use of AI Tool

I used ChatGPT, Google Gemini and AI-assisted database tools as learning and development aids during this project. These tools helped me understand database concepts, review SQL, JavaScript and Python syntax, troubleshoot errors encountered during implementation and organize the technical documentation. However, I personally configured the PostgreSQL and MongoDB environments, executed the scripts, conducted the experiments and verified the results on my local computer. I also reviewed the AI-generated suggestions against the actual database outputs and documented the results using screenshots captured during the experiments. The final analysis and conclusions reflect my understanding of the work carried out.